



GLIDER MODEL

FINAL REPORT

SAVJOT SINGH CHEEMA

INDIA

July 8, 2017

Summary

Detailed mathematical model of glider (ATS-1 Ardhra) is studied in this report. The trajectory and flight of glider is predicted in this model. The model starts with free body diagram of glider. Two differential equations are used to study the flight of glider. The solutions are calculated using Euler's method. First, both equations are solved individually. Some initial values are approximated and Python code is generated for both differential equations. Calculated results of individual differential equations of velocity and angle help in studying changes in flight of glider with respect to time. However interaction of both variables with each other is unknown.

To solve this problem, extension of model is done through combined system of differential equations. It gives a more realistic view on trajectory of glider. Again Python program is made for combined set of differential equations. Within this code two differential equations for distance covered in horizontal and vertical direction are included. Addition of two more differential equation of distance help in understanding the trajectory of glider in better way. Linearisation of system around equilibrium point is done and the long term behaviour of glider is understood through equilibrium points.

The model is then extended beyond fixed initial conditions and results are calculated for different initial conditions. This helps in understanding behaviour of glider under extreme conditions. From calculations, it becomes clear that under which conditions glider will glide efficiently. After this, report focuses on a specific problem. In the end, results are concluded.

Contents

Summary	3
List of variables	7
1 Introduction	8
2 Basic Glider Model	9
2.1 Calculation of Variables	10
2.1.1 Mass of Glider, m	10
2.1.2 Typical Speed of Glider	10
2.2 Differential Equation for Angle, θ	11
2.2.1 Calculation of Lift Coefficient, C_L	11
2.2.2 Simulation of $\frac{d\theta}{dt}$ in Python	11
2.2.3 Error Estimation	12
2.2.4 Calculating θ_{eq}	13
2.3 Differential Equation for Velocity, V	13
2.3.1 Calculation of Drag Coefficient, C_D	13
2.3.2 Simulation of $\frac{dV}{dt}$ in Python	14
2.3.3 Error Estimation	14

2.4	Results	16
3	System of Differential Equation	18
3.1	Simulation of the system for $V(t)$ and $\theta(t)$	18
3.2	Trajectory of Glider	19
3.3	Calculation of Equilibrium Point	20
3.4	Linearisation	21
3.4.1	Jacobian matrix	21
3.4.2	Characteristic Equation	21
3.4.3	Calculation of Eigen Values	22
3.5	Results	22
4	Extension of Model	24
4.1	Recalculation of variables	24
4.1.1	Mass and Typical Speed	24
4.1.2	Recalculation of Lift Coefficient, C_L	24
4.1.3	Recalculation of Drag Coefficient, C_D	25
4.2	Simulation with different initial conditions	25
4.2.1	Simulation for different initial conditions of $\theta(t)$	25
4.2.2	Simulation for different initial conditions of $V(t)$	26
4.3	Results	27
5	Formulation and Solution of Specific Problem	29
5.1	Problem Formulation	29
5.2	Gliding Time	30
5.3	Horizontal Distance Covered	30
5.4	Final Velocity	31

5.5	Total Change in Angle	32
5.6	Maximum Height	32
6	Conclusions	34
	Bibliography	36
A	Python Code for $\frac{d\theta}{dt}$	37
B	Python Code for $\frac{dV}{dt}$	39
C	Python Code for System of Two Differential Equations	41
D	Python Code for System of Four Differential Equations	43
E	Python Code for Different Initial Conditions of $\theta(t)$	46
F	Python Code for Different Initial Conditions	49

List of variables

m : Mass of loaded glider in kilograms [Kg]
 V : Velocity of glider in metre per second [m/s]
 C_L : Lift Coefficient [Kg/m]
 C_D : Drag Coefficient [Kg/m]
 θ : Angle with horizon in radians [rad]
 t : Time in seconds [s]
 g : Gravitational acceleration [9.81 m/s^2]
 x : Distance covered by glider in horizontal direction [m]
 y : Distance covered by glider in vertical direction [m]

Chapter 1

Introduction

A glider is a heavier-than-air aircraft that is supported in flight by the dynamic reaction of the air against its lifting surfaces, and whose free flight does not depend on an engine. [1] Gliders are principally used for the air sports of gliding, hang gliding and paragliding. However some spacecraft have been designed to descend as gliders and in the past military gliders have been used in warfare. Thus, they have very important application in various fields, especially in military and as spacecrafts. Therefore the study of such gliders is very important. But how to study the flight of such gliders? Which particular glider should be chosen to start with? How precisely the mathematical modelling of trajectory of glider must be done? What must be the conditions for efficient working of glider?

All these questions are answered in this report. However the model in this report is not completely realistic. Various variables are kept constant which vary in real world. In chapter 2, mathematical modelling is started with two differential equations. The results are calculated using Euler's Method with minimum error. However these solutions are calculated for each differential equation separately. The model is extended in next chapter. In Chapter 3, modelling is done using a system of differential equations which helps in predicting trajectory of glider. Long term behaviour of glider is studied with the help of equilibrium points. In Chapter 4, behaviour of glider for different initial conditions is noticed. Solutions are calculated for efficient working of glider. In Chapter 5, solutions are calculated for one specific problem. Results are concluded at last.

Chapter 2

Basic Glider Model

To start mathematical modelling of glider, first, one particular glider is choosen. Hindustan Ardhra is choosen for the modelling. All the quantitative information is collected. The Hindustan Ardhra was a glider designed in India for pilot training by the government's Civil Aviation Department in the late 1970s as the ATS-1 Ardhra. It was a two-seat aircraft of conventional configuration and wooden construction. The specifications of this glider [2] are used in this report and thus its flight is studied.

Assume a glider flies with (scalar) velocity V and under an angle θ with the horizon. Distances covered in horizontal and vertiical direction are takes as x and y respectively. The glider has no engine, and there are three forces working on the glider: gravity, drag and lift. In this simple model, both the drag and the lift are proportional to V^2 . All the forces acting on it are shown in a free body diagram in Figure 2.1.

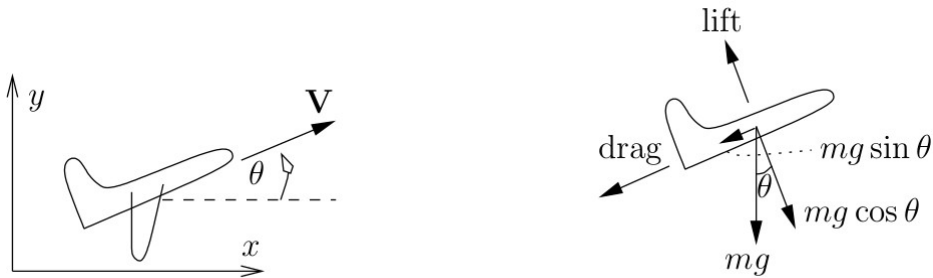


Figure 2.1: **Glider Model:** Free Body Diagram of Glider.

Following two equations 2.1 and 2.2 show differential equations for this simple

model:

$$m \frac{dV}{dt} = -C_D V^2 - mg \sin(\theta) \quad (2.1)$$

$$mV \frac{d\theta}{dt} = +C_L V^2 - mg \cos(\theta) \quad (2.2)$$

where

m = Mass of loaded glider in kilograms [Kg]

V = Velocity of glider in metre per second [m/s]

C_L = Coefficient of lift [Kg/m]

C_D = Coefficient of drag [Kg/m]

θ = Angle with horizon in radians [rad]

t = Time in seconds [s]

g = Gravitational acceleration [m/s^2]

In aerodynamics the so called angle of attack is the angle between the wings and the oncoming airflow. We assume in this model that there is no wind and that the angle of attack is constant, so parameters C_D and C_L are assumed to have constant values.

2.1 Calculation of Variables

The values of various variables are calculated according to specifications of glider ATS-1 Ardhril [2].

2.1.1 Mass of Glider, m

Mass of empty glider = 328 Kg

Number of passengers = 2

Assume mass of each passenger = 71 Kg

Mass of loaded glider, m = 328 + 71 * 2 = 470 Kg

2.1.2 Typical Speed of Glider

Typical speed = 100 Km/h = 27.78 m/s

2.2 Differential Equation for Angle, θ

The differential equation for θ is considered. First the typical speed of the glider is taken as the value of V . The differential equation for θ is

$$\lim_{\Delta t \rightarrow 0} \frac{\Delta \theta}{\Delta t} = \frac{d\theta}{dt} = \frac{C_L V^2 - mg \cos(\theta)}{mV}$$

2.2.1 Calculation of Lift Coefficient, C_L

We know that

$$\begin{aligned} \lim_{\Delta t \rightarrow \infty} \frac{\Delta \theta}{\Delta t} &= \frac{d\theta}{dt} = 0 \\ \frac{C_L V^2 - mg \cos(\theta)}{mV} &= 0 \end{aligned}$$

Put known values into the equation,

$$\frac{C_L 27.78^2 - 470 * 9.81 \cos(\theta)}{470 * 27.78} = 0$$

$$C_L = \frac{470 * 9.81}{27.78^2} \cos(\theta) \quad (2.3)$$

The range for the value of C_L is directly dependent upon the value of θ . So, let the range of θ be $[-\frac{\pi}{3}, \frac{\pi}{3}]$. To keep the glider in flight for maximum time the value of C_L must be maximum. Thus θ should be $\frac{\pi}{3}$. Maximum value of C_L is calculated.

$$C_L = 3Kg/m$$

2.2.2 Simulation of $\frac{d\theta}{dt}$ in Python

The solutions of the differential equation are approximated using Euler's method. Of course calculations by hand are inefficient. The method is very suitable to implement in a computer program: much faster and probably more accurate. So in this section, Python is used to simulate the same solutions. Initially let $\theta(0) = \frac{\pi}{6}$. The solutions for $\Delta t = 1$ sec are calculated. The stepsize is investigated later.

The code generated in Python for $\frac{d\theta}{dt}$ is included in Appendix A. Figure 2.2 shows the graph of rate of change of angle with time.

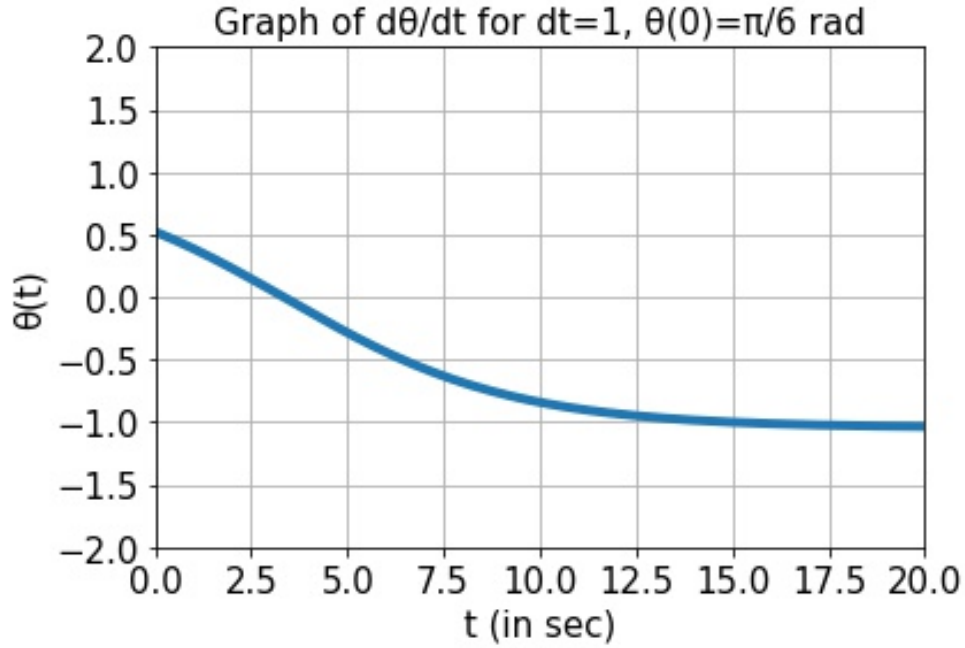


Figure 2.2: **Graph of θ vs t .**

2.2.3 Error Estimation

In the previous subsection the approximate solutions for $\Delta t=1$ were calculated. In this section, the stepsize is investigated further. The smaller Δt is taken, the better approximation becomes and it will look more and more like the exact solution.

The largest differences in the value of θ occur around 3 seconds according to Figure 2.2. So when the angle at 3 seconds is to be studied, the stepsize is very important. Estimated error should be less than 0.01 rad.

The code generated in Python for $\frac{d\theta}{dt}$ is included in Appendix A. Table 2.1 shows estimated values of error.

Stepsize $\Delta t(\text{seconds})$	Value of Angle at $t=3$ sec $\theta(t)$	Estimated Error $\Delta\theta$ (radians)
1	0.081	
0.5	0.067	-0.014
0.25	0.060	-0.007

Table 2.1: **Error Estimation** Error estimated when stepsize is halved

Thus stepsize $\Delta t=0.25$ sec is small enough for further modelling.

2.2.4 Calculating θ_{eq}

The typical angle the glider flies under on a long undisturbed part of the flight is θ_{eq} .

For $\Delta t=0.25$ sec and $\theta(0) = \frac{\pi}{6}$ run the simulated program. The code generated in Python for $\frac{d\theta}{dt}$ is included in Appendix A. From the output we get $\theta_{eq} = -1.045$ rad.

2.3 Differential Equation for Velocity, V

The differential equation for V is considered. θ is taken constant and Euler's method for the differential equation for V is implemented. The typical angle the glider flies under on a long undisturbed part of the flight is used. A few different values for V(0) can be chosen, however minimum value of V is 16.95 m/s (check specification [2] for stall speed = 61 km/h = 16.85 m/s).

$$\frac{dV}{dt} = \frac{-C_D V^2 - mg \sin(\theta)}{m}$$

2.3.1 Calculation of Drag Coefficient, C_D

We know that

$$\lim_{\Delta t \rightarrow \infty} \frac{\Delta V}{\Delta t} = \frac{dV}{dt} = 0$$

$$\frac{-C_D V^2 - mg \sin(\theta)}{m} = 0$$

Put known values into the equation,

$$\frac{-C_D V^2 - 470 * 9.81 \sin(-1.045)}{470} = 0$$

$$C_D = \frac{470 * 9.81}{V^2} \sin(1.045)$$

The value of C_D depends upon V. The minimum value of V is 16.95 m/s. Thus, maximum value of C_D is

$$C_D = \frac{470 * 9.81}{16.95^2} \sin(1.045) = 13.90 Kg/m$$

2.3.2 Simulation of $\frac{dV}{dt}$ in Python

The solutions of the differential equation are approximated using Euler's method. Of course calculations by hand are inefficient. The method is very suitable to implement in a computer program: much faster and probably more accurate. So in this section, Python is used to simulate the same solutions.

The code generated in Python for $\frac{dV}{dt}$ is included in Appendix B. The value of $V(t)$ must be greater than 16.95 m/s [2]. Thus $V(0)$ is taken to be 50 m/s. Initially solutions are calculated for $\Delta t = 1$ sec. Figure 2.3 show results through graph.

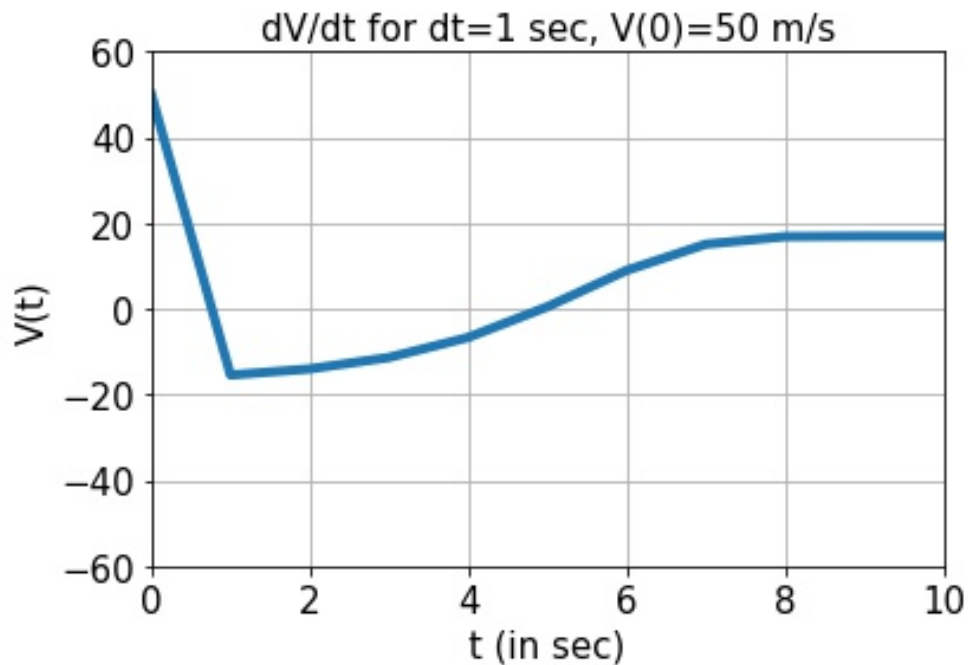


Figure 2.3: Graph of V vs t.

This stepsize shows irrelevant result. So decrease the stepsize to half, i.e. $\Delta t = 1/2$ sec. Figure 2.4 show relevant results through graph.

2.3.3 Error Estimation

In the previous subsection the approximate solutions for $\Delta t=1/2$ were calculated. In this section, the stepsize is investigated further. The smaller Δt is

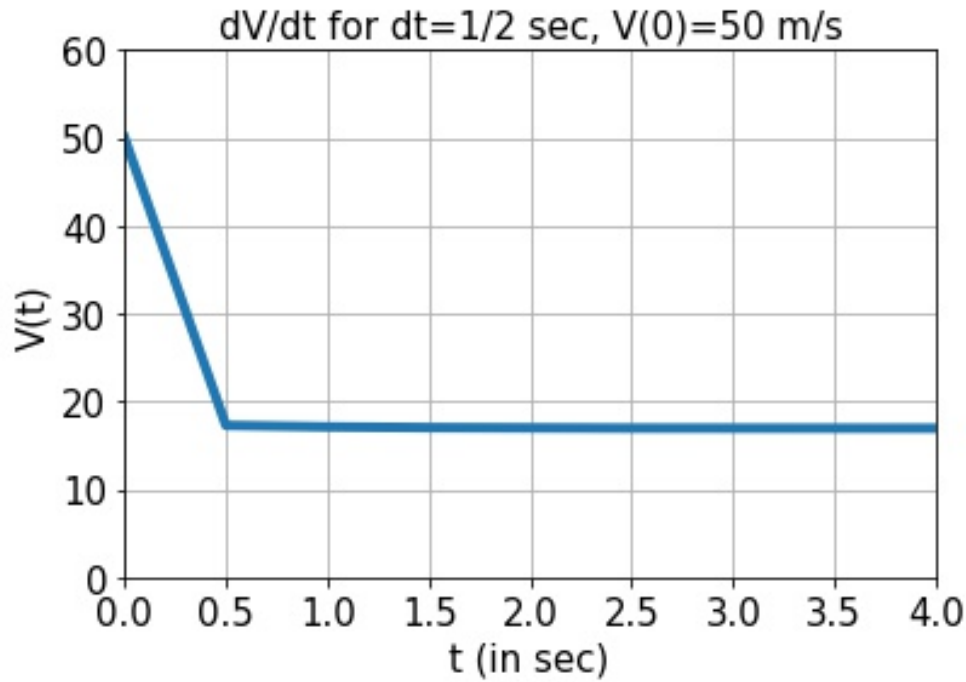


Figure 2.4: **Graph of V vs t.**

taken, the better approximation becomes and it will look more and more like the exact solution.

From Figure 2.4 , the largest differences in the value of V occur around 0.25 or 1/4 seconds. So when the velocity at 0.25 seconds is to be studied, the step-size is very important. Estimated error should not be taken more than 0.1 m/s.

The code generated in Python for $\frac{dV}{dt}$ is included in Appendix B. Values of estimated errors are shown in table 2.2. The estimated error for $dt=1/32$ is

Stepsize $\Delta t(\text{seconds})$	Value of Velocity at $t=0.25$ sec $V(t)$	Estimated Error ΔV (m/s)
1/4	31.597	
1/8	34.686	3.089
1/16	35.720	1.034
1/32	36.163	0.443

Table 2.2: **Error Estimation** Error estimated when stepsize is halved

less than 0.1 m/s. Thus stepsize $\Delta t=1/32$ sec is small enough for further modelling.

2.4 Results

The graph of the solution of the differential equation for θ is presented in figure 2.5. The graph shows the change in value of angle of glider with horizon with respect to time. The angle changes continuously with time. Initially, the angle of glider decreases with time at fast rate and becomes constant after sometime (after 15 seconds to be precise).

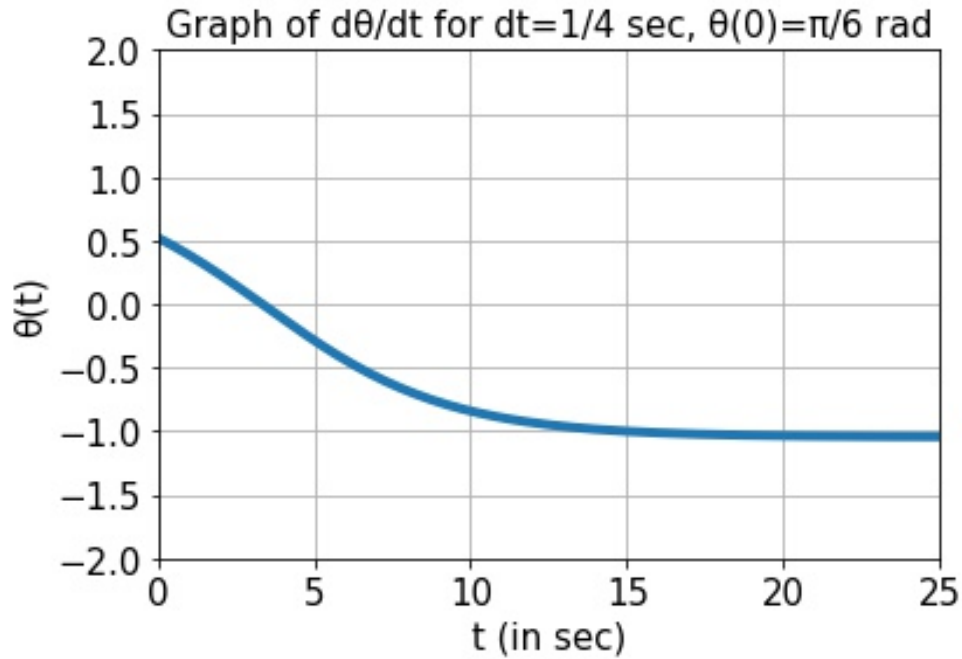


Figure 2.5: Graph of θ vs t .

The graph of the solution of the differential equation for V is presented in figure 2.6. This graph shows the change in velocity of glider with time. Initially, the velocity decreases at fast rate and becomes constant after sometime.

The trajectory and flight of the glider is studied with the help differential equations. The values of some variables like mass, typical speed, initial values of speed and angle are assumed and calculated, and thus the differential equations are simplified. The equilibrium points of each equation is calculated separately. After this we ask ourselves the question: How long does it take to

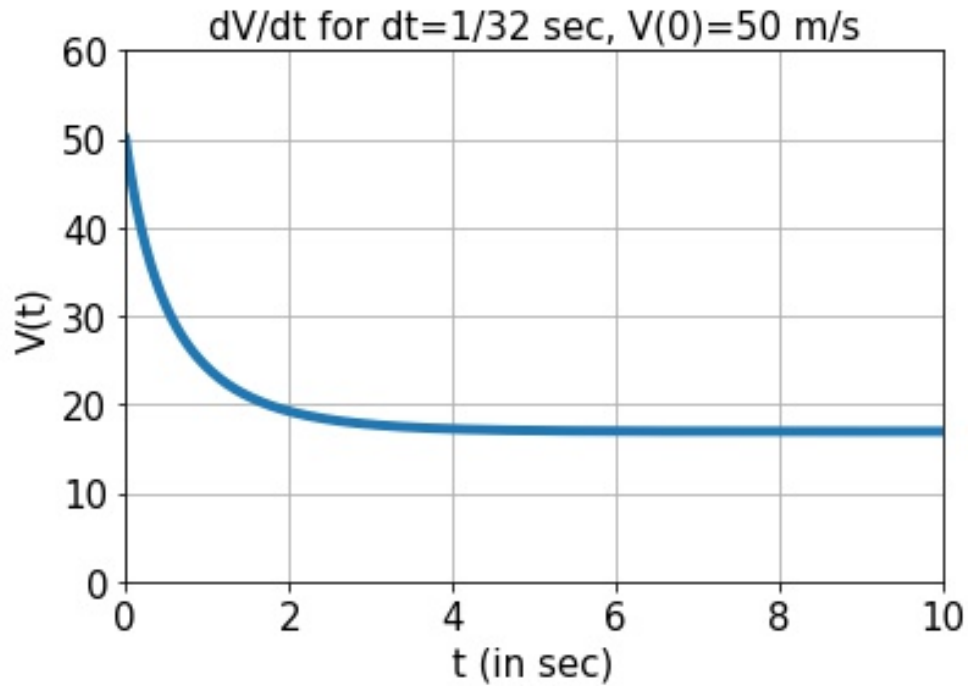


Figure 2.6: **Graph of V vs t .**

reach the equilibrium point? To answer this question, the Euler's method is implemented and solutions are simulated on a computer. Right stepsize is calculated and the error is estimated. The solutions are calculated for relevant stepsize and results are generated through graphs made using Python code. Graph of θ vs t shows the angle of glider decreases continuously until it reaches θ_{eq} . Similarly the graph of V vs t is generated and results are studied. However these equations are solved separately in this chapter. The influence of one equation on other is to be studied in next chapter where we will work on system of differential equations.

Chapter 3

System of Differential Equation

In previous chapter the solutions for each differential equation were approximated separately. Thus the influence of velocity on angle of glider with respect to time or vice versa was unknown. In this chapter we will start with system of differential equations and approximate solutions for both differential equations using Euler's equation. In this way the trajectory of glider can be predicted.

3.1 Simulation of the system for $V(t)$ and $\theta(t)$

One set of initial conditions $V(0)$ and $\theta(0)$ is chosen. Euler's method for the system is implemented, where both V and θ change in time. Since different stepsizes were decided for the calculations of the separate $V(t)$ and $\theta(t)$, for this system the calculations are done with the minimum of the two stepsizes.

Thus,

$$V(0) = 50 \text{ m/s}$$

$$\theta(0) = \pi / 6 \text{ rad}$$

$$\Delta t = 1/32 \text{ s}$$

The code generated in Python for $\frac{dV}{dt}$ and $\frac{d\theta}{dt}$ is included in Appendix C. The graph of the solution of the differential equation for V and θ is presented in Figure 3.2. It shows the change in value of velocity and angle (with horizon) of glider with respect to time.

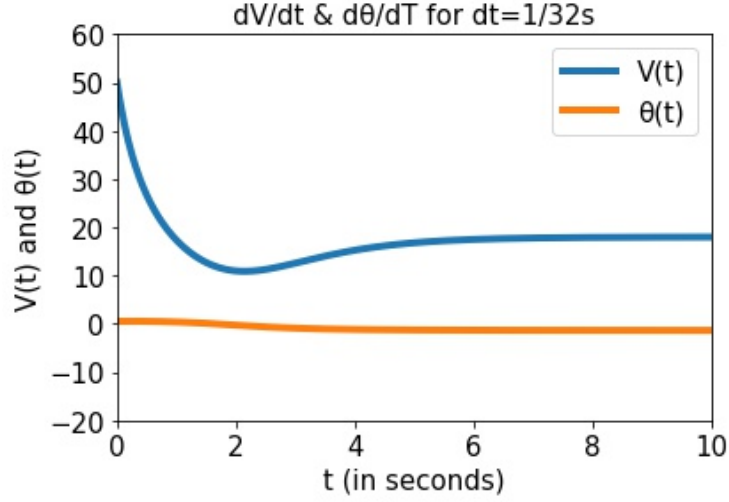


Figure 3.1: V vs t and θ vs t .

3.2 Trajectory of Glider

After the solutions of the system for $V(t)$ and $\theta(t)$ are simulated, it is time to predict the trajectory of the glider. The trajectory of glider through the sky is described by the horizontal position $x(t)$ and the vertical position, or height $y(t)$. Variables x and y are described by first-order differential equations 3.1 and 3.2:

$$\frac{dx}{dt} = f_x(V, \theta) = V \cos(\theta) \quad (3.1)$$

$$\frac{dy}{dt} = f_y(V, \theta) = V \sin(\theta) \quad (3.2)$$

The expressions for $f_x(V, \theta)$ and $f_y(V, \theta)$ are derived using Figure 2.1.

Now the two differential equations for x and y are included into simulation program. In this way, four dependent variables V , θ , x and y are solved at the same time. Thus, trajectories are plotted as in Figure 3.2. Python program is included in Appendix D. Let initial height of glider be 1000m. The vertical height of glider increase for sometime and then start decreasing. The horizontal distance travelled by glider keep on increasing until it reaches ground.

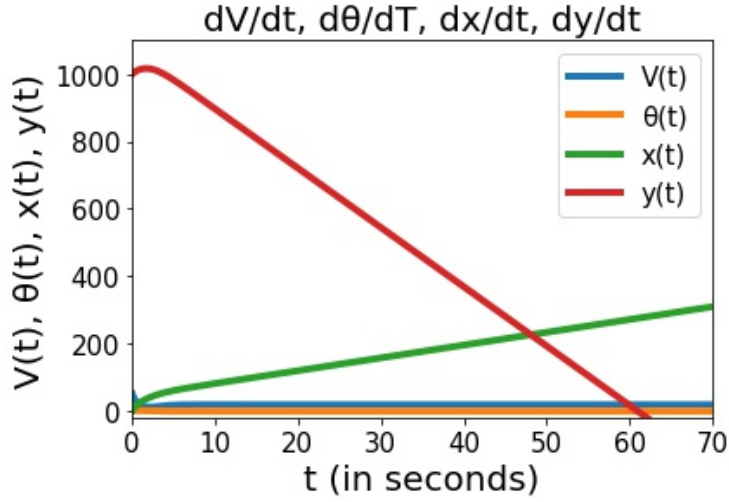


Figure 3.2: **Graph of 4 Dependent Variables** $V(t)$, $\theta(t)$, $x(t)$ and $y(t)$

3.3 Calculation of Equilibrium Point

At equilibrium point

$$\lim_{\Delta t \rightarrow \infty} \frac{\Delta V}{\Delta t} = \frac{dV}{dt} = 0$$

$$\frac{-C_D V^2 - mg \sin(\theta)}{m} = 0$$

$$\sin(\theta) = \frac{-C_D V^2}{mg}$$

Also,

$$\lim_{\Delta t \rightarrow \infty} \frac{\Delta \theta}{\Delta t} = \frac{d\theta}{dt} = 0$$

$$\frac{C_L V^2 - mg \cos(\theta)}{mV} = 0$$

$$\cos(\theta) = \frac{C_L V^2}{mg}$$

Thus,

$$\tan(\theta) = -\frac{C_D}{C_L}$$

$$\theta_{eq} = -\frac{13.90}{3} = -1.358 \text{ rad}$$

$$V_{eq} = \sqrt{\frac{mg \cos(\theta)}{C_L}} = 18 \text{ m/s}$$

The equilibrium point (V_{eq}, θ_{eq}) is $(18, -1.358)$.

3.4 Linearisation

A mathematical model for the trajectory of glider is the following system of differential equations.

$$\begin{cases} \frac{dV}{dt} = \frac{-C_D V^2 - mg \sin(\theta)}{m}, \\ \frac{d\theta}{dt} = \frac{+C_L V^2 - mg \cos(\theta)}{mV} \end{cases}$$

In this section the system is linearised in the neighbourhood of an equilibrium point. With the linearisations, some of the results can be predicted analytically from the parameters. The system of linearised differential equations can be rewritten into a different form by using vectors, that is, in matrix-vector form:

$$\begin{bmatrix} \frac{dV}{dt} \\ \frac{d\theta}{dt} \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial V}|_{(V_0, \theta_0)} & \frac{\partial f_1}{\partial \theta}|_{(V_0, \theta_0)} \\ \frac{\partial f_2}{\partial V}|_{(V_0, \theta_0)} & \frac{\partial f_2}{\partial \theta}|_{(V_0, \theta_0)} \end{bmatrix} \begin{bmatrix} V - V_0 \\ \theta - \theta_0 \end{bmatrix}$$

Here, (V_0, θ_0) is an equilibrium point. Now we define $\vec{X}(t)$ and $\vec{X}_0$ and matrix $J(\vec{X}_0)$ as

$$\vec{X}(t) = \begin{bmatrix} V(t) \\ \theta(t) \end{bmatrix}, \vec{X}_0 = \begin{bmatrix} V_0 \\ \theta_0 \end{bmatrix}, \text{ and } J(\vec{X}_0) = \begin{bmatrix} \frac{\partial f_1}{\partial V}|_{(V_0, \theta_0)} & \frac{\partial f_1}{\partial \theta}|_{(V_0, \theta_0)} \\ \frac{\partial f_2}{\partial V}|_{(V_0, \theta_0)} & \frac{\partial f_2}{\partial \theta}|_{(V_0, \theta_0)} \end{bmatrix}$$

3.4.1 Jacobian matrix

Matrix $J(\vec{X}_0)$ is a matrix with elements that do not depend on time t . It is called the Jacobian matrix and plays an important role in our analysis of the system.

$$J(\vec{X}_0) = \begin{bmatrix} \frac{\partial f_1}{\partial V}|_{(V_0, \theta_0)} & \frac{\partial f_1}{\partial \theta}|_{(V_0, \theta_0)} \\ \frac{\partial f_2}{\partial V}|_{(V_0, \theta_0)} & \frac{\partial f_2}{\partial \theta}|_{(V_0, \theta_0)} \end{bmatrix} = \begin{bmatrix} \frac{-2C_D V}{m} & -g \cos(\theta) \\ \frac{C_L}{m} + \frac{g \cos(\theta)}{V^2} & \frac{g \sin(\theta)}{V} \end{bmatrix}$$

3.4.2 Characteristic Equation

Let λ be the eigen value of Jacobian Matrix. The characteristic equation can be derived using following relation (for detailed study of characteristic equations and eigen values refer[3]):

$$\left| \begin{bmatrix} \frac{\partial f_1}{\partial V}|_{(V_0, \theta_0)} & \frac{\partial f_1}{\partial \theta}|_{(V_0, \theta_0)} \\ \frac{\partial f_2}{\partial V}|_{(V_0, \theta_0)} & \frac{\partial f_2}{\partial \theta}|_{(V_0, \theta_0)} \end{bmatrix} - \begin{bmatrix} \lambda & 0 \\ 0 & \lambda \end{bmatrix} \right| = 0$$

$$\left| \begin{array}{cc} \frac{-2C_D V}{m} - \lambda & -g \cos(\theta) \\ \frac{C_L}{m} + \frac{g \cos(\theta)}{V^2} & \frac{g \sin(\theta)}{V} - \lambda \end{array} \right| = 0$$

Solving it we get final characteristic equation as

$$\lambda^2 + 1.6\lambda + 0.594 = 0$$

3.4.3 Calculation of Eigen Values

The characteristic equation is an quadratic equation. Solution of this equation will give two eigen values of matrix.

$$\lambda^2 + 1.6\lambda + 0.594 = 0$$

$$\lambda = \frac{-1.6 + \sqrt{1.6^2 - 4 * 0.594}}{2}, \frac{-1.6 - \sqrt{1.6^2 - 4 * 0.594}}{2}$$

We get $\lambda_1 = -1.0145$ and $\lambda_2 -0.5855$

3.5 Results

An equilibrium point $\vec{X}_0$ is called a stable node if the Jacobian matrix $J(\vec{X}_0)$ has two negative eigenvalues: all solutions that start near the equilibrium point stay near the equilibrium point. An example of a phase portrait is shown in figure 3.3. Thus the equilibrium point (18, -1.358) is stable.

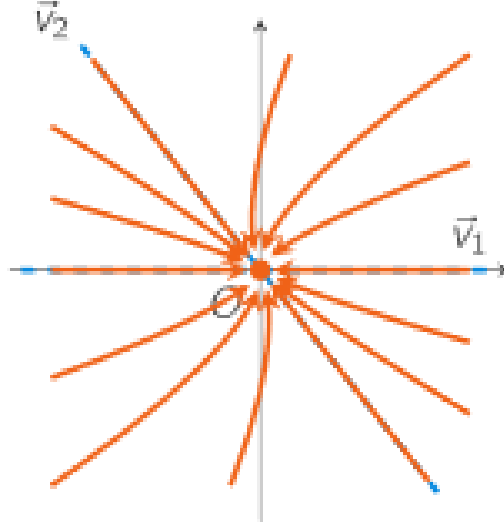


Figure 3.3: **Stable Node:** $\lambda_1 < 0$, $\lambda_2 < 0$, ($\vec{v}_1$ and $\vec{v}_2$ are vectors of Jacobian Matrix).

In this chapter, the interaction of velocity and angle of glider with respect to time is studied through combined system of differential equations. Addition of two more differential equation of x and y help in understanding the trajectory of glider. The long term behaviour of glider in sky is understood by calculating equilibrium points. System of differential equation is written in matrix form and linearisation of system around stability point is done. This determines that how the system of differential equations behave in the neighbourhood of the equilibrium points. By calculation of eigen values, the type of equilibrium becomes clear. Therefore the the behaviour of glider is studied after it reaches stability.

In next chapter, the model is extended, and whole modelling cycle is recalculated.

Chapter 4

Extension of Model

In previous models, initial values were kept constant and solutions were calculated for one particular initial condition. In this chapter, the model will be extended and simulations will be done when there are different initial conditions. Thus, modelling cycle will repeat from start to end. First, lift coefficient, drag coefficients and other variables are to be calculated again.

4.1 Recalculation of variables

4.1.1 Mass and Typical Speed

The mass of glider remains same because this particular glider can carry two persons. Thus weight of glider will be sum of weight of empty glider and weight of two persons (refer section 2.1).

$$m = 470 \text{ KG}$$

Now the typical speed changes to equilibrium speed calculated in previous model.

$$V_{eq} = 18 \text{ m/s}$$

4.1.2 Recalculation of Lift Coefficient, C_L

As calculated before in section 2.2.1, the value of C_L is calculated in same way except some values are changed for accurate results.

Therefore by equation 2.3

$$C_L = \frac{470 * 9.81}{18^2} \cos(\theta)$$

$$C_L = \frac{470 * 9.81}{18^2} \cos(\pi/2.313)$$

$$C_L = 3.002$$

There is no significant change in value of lift coefficient.

4.1.3 Realcuation of Drag Coefficient, C_D

The stall speed of glider choosen in this report is fixed. Therefore, the value of drag coefficient will remian same (refer section 2.3.1).

4.2 Simulation with different initial conditions

Now that all the required variables are recalculated; it is time to simulate glider flight with different initial conditions.

4.2.1 Simulation for different initial conditions of $\theta(t)$

Figure 4.1 shows $\theta(t)$ vs time graph for different values of $\theta(0)$. The code generated in Python for $\frac{d\theta}{dt}$ is included in Appendix E.

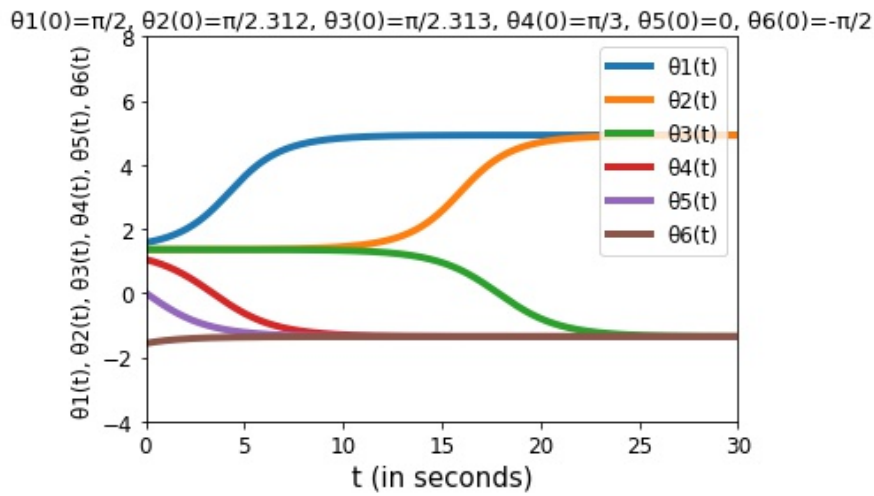


Figure 4.1: **Graph of $\theta(t)$ vs time:** For different initial conditions

By graph, the maximum value of $\theta(0)$ up to which solutions are relevant is $\theta = \pi/2.313$.

4.2.2 Simulation for different initial conditions of $V(t)$

Now, the simulation is done and velocity-time graph is generated for different initial values of velocity. The code generated in Python for $\frac{d\theta}{dt}$ and $\frac{dV}{dt}$ is included in Appendix F.

Figure 4.2 shows graph for different initial values of velocity when $\theta(0) = \pi/3$.

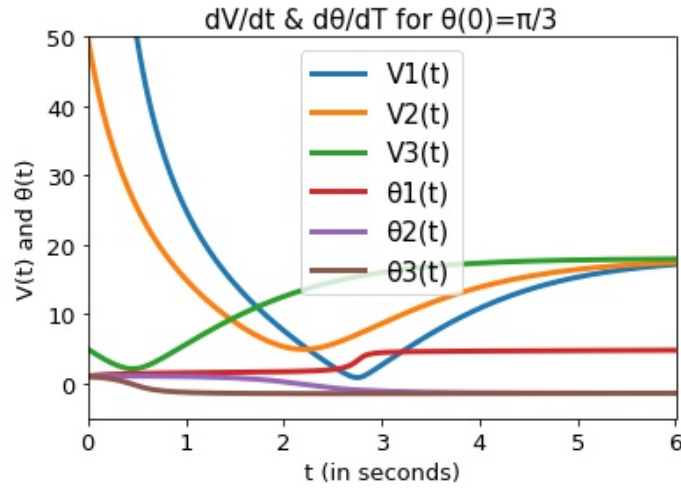


Figure 4.2: **Graph of $\theta(t)$ and $V(t)$ vs time:** $V1(0)=500\text{m/s}$, $V2(0)=50\text{m/s}$, and $V3(0)=5\text{m/s}$

Figure 4.3 shows graph for different initial values of velocity when $\theta(0) = 0$.

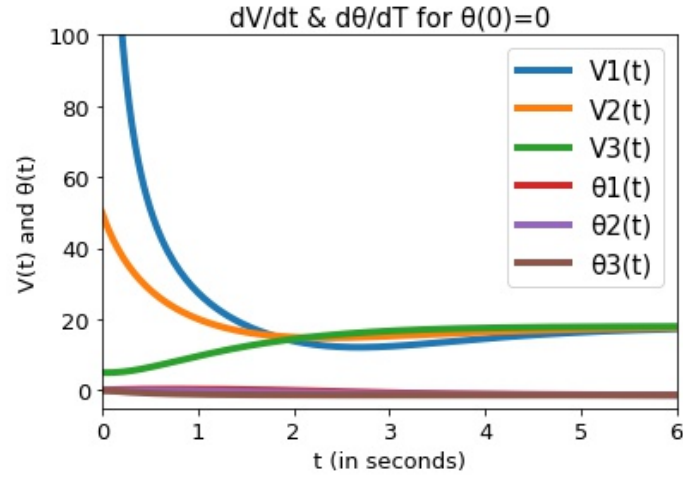


Figure 4.3: **Graph of $\theta(t)$ and $V(t)$ vs time:** $V_1(0)=500\text{m/s}$, $V_2(0)=50\text{m/s}$, and $V_3(0)=5\text{m/s}$

Figure 4.4 shows graph for different initial values of velocity when $\theta(0) = -\pi/3$.

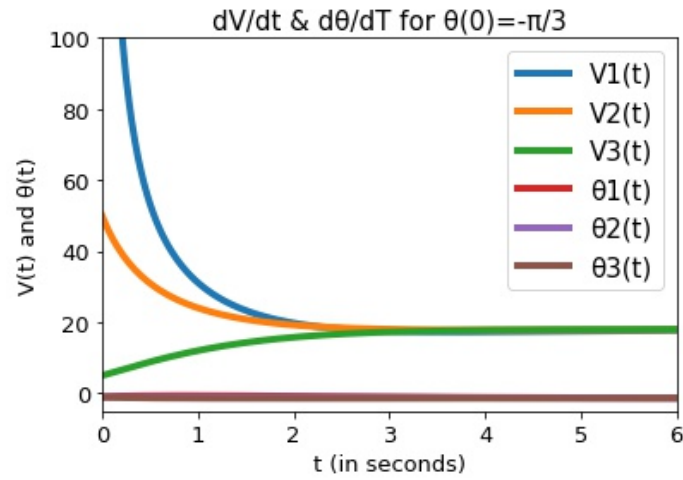


Figure 4.4: **Graph of $\theta(t)$ and $V(t)$ vs time:** $V_1(0)=500\text{m/s}$, $V_2(0)=50\text{m/s}$, and $V_3(0)=5\text{m/s}$

4.3 Results

Follwing conclusions can be made from previous study:

- It can be concluded from figure 4.1 that the flight of glider can be started

having its nose tip is within angular range $[\frac{\pi}{2.313}, \frac{-\pi}{2}]$. More the glider is declined from front, more stable it will be.

- The glider will reach its equilibrium state when its velocity will be constant. From section 4.2.2 it can be stated that final velocity (stable state) of glider is independent of initial velocity. No matter if initial velocity is 5 m/s or 500 m/s, final velocity will be the same.

In next chapter, results are calculated for the specific problem.

Chapter 5

Formulation and Solution of Specific Problem

Now that this model has been extended and remodelled again, it is time to calculate results for specific problem.

5.1 Problem Formulation

The problem is "The flight of ATS-1 glider starts from the hill top. The height of hill from ground level is 5000 m. Glider is pushed from hill top so that it start gliding. Assume that this initial push gives the glider an initial velocity of 8 m/s."

To calculate:

- What is the total time of gliding?
- How far does the glider goes from hill?
- What is the velocity at which glider covers most of the distance during its flight?
- Total change in angular orientation of glider.
- If the height of glider increase more than the height of hill top? If it does, what is the maximum height?

5.2 Gliding Time

The initial time i.e. $t = 0$ sec is the time when glider is given push. Therefore V_{in} or $V(0) = 8$ m/s. Initial angle of glider with horizontal is zero. Euler's method is used (refer section 3.2) and results are calculated through python program (refer code: Appendix D).

Total gliding time is the time for which glider remains in air. Figure 5.1 shows vertical height vs time graph when $\theta(0) = 0$ and initial vlocity is 8 m/s. By graph, glider reaches ground in 286 seconds. So, the gliding time is 4 minutes and 46 seconds.

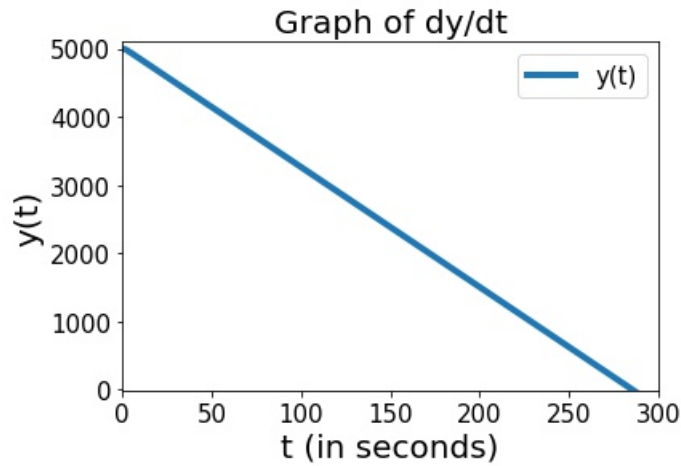


Figure 5.1: **Height (y) vs Time (t)**

5.3 Horizontal Distance Covered

Figure 5.2 shows graph of vertical and horizontal distance vs time. The horizontal distance covered by glider before coming to ground is 1092 m. Thus it is 1 Kilometre and 92 metres away from the hill.

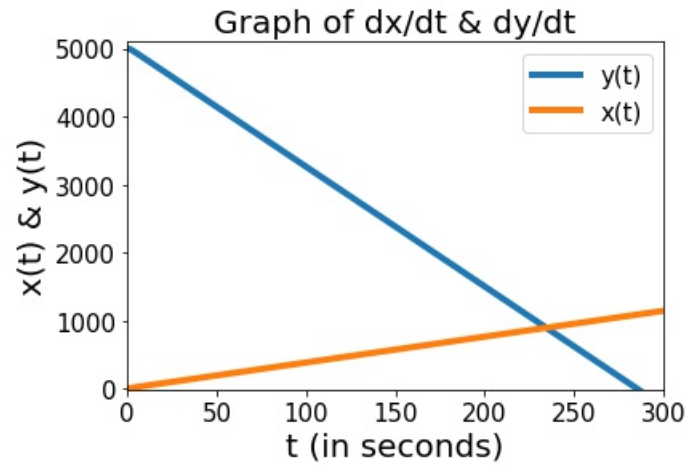


Figure 5.2: **Height (y) and Distance (x) vs Time (t)**

5.4 Final Velocity

Figure 5.3 shows velocity vs time graph when initial vlocity is 8 m/s. The velocity of glider first increases and then becomes constant. Its final velocity is 18 m/s.

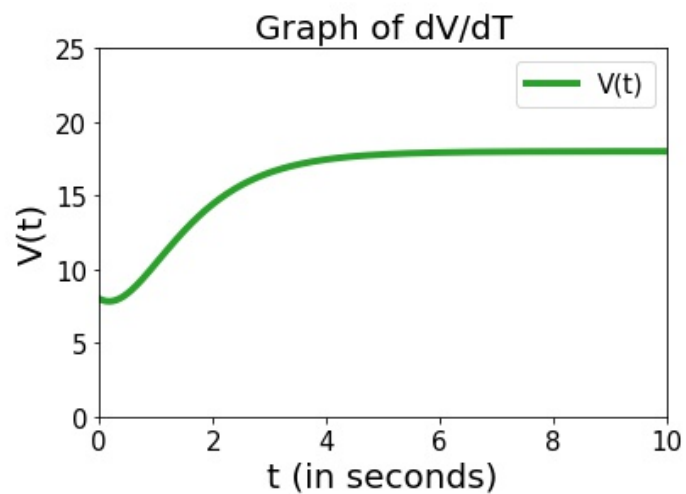


Figure 5.3: **Velocity vs Time**

5.5 Total Change in Angle

Figure 5.4 shows θ vs time graph. The angle of glider changes from zero to -1.358 radians (or 77.8°).

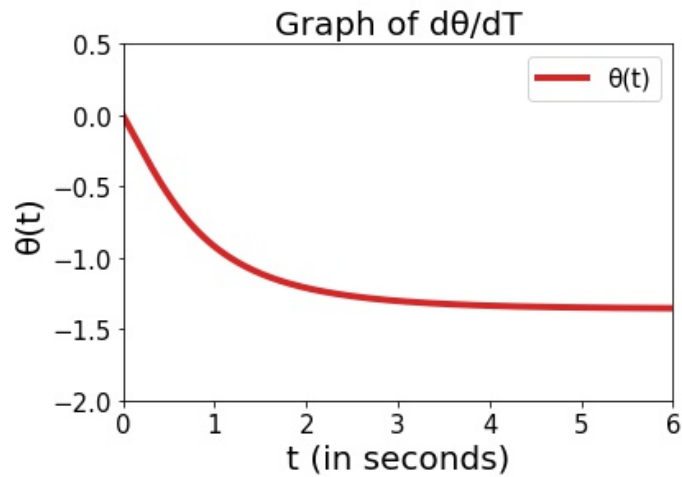


Figure 5.4: **Angle vs Time**

5.6 Maximum Height

The height of glider decreases non- uniformly. Figure 5.5 shows vertical height ($=y$) vs time graph when $\theta(0) = 0$ and initial vlocity is 8 m/s.

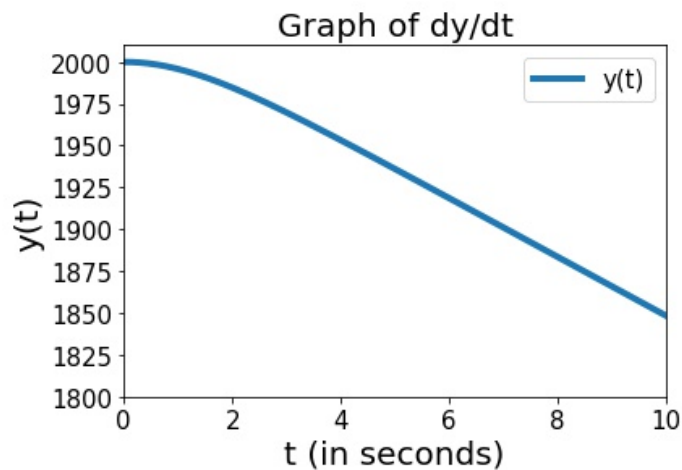


Figure 5.5: **Height vs Time**

The height will increase than its initial height only when initial velocity is high. For an example, when initial velocity is 200 m/s maximum height is 5020 m (see figure 5.6).

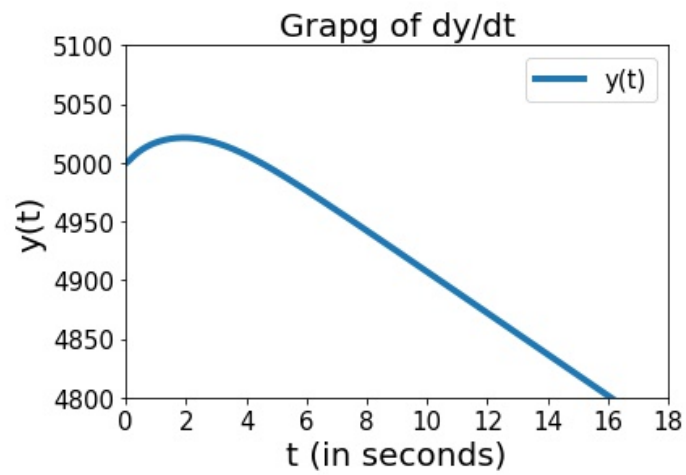


Figure 5.6: **Height vs Time**

Chapter 6

Conclusions

The study of gliders is very important. But how to study the flight of such gliders? What must be the mathematical model to predict flight of glider? How precisely the mathematical modelling of trajectory of glider must be done? The trajectory and flight of the glider is studied with the help differential equations. The values of some variables are calculated and some are assumed, and thus the differential equations are simplified. The equilibrium points of each equation is calculated separately. After this we ask ourselves the question: How long does it take to reach the equilibrium point? To answer this question, the Euler's method is implemented and solutions are simulated on a computer. Right stepsize is calculated and the error is estimated. The solutions are calculated for relevant stepsize and results are generated through graphs made using Python code. Graph of θ vs t shows the angle of glider decreases continuously until it reaches θ_{eq} . Similarly the graph of V vs t is generated and results are studied. However both differential equations are solved separately.

After the solutions for each differential equation were approximated separately, the influence of velocity on angle of glider with respect to time or vice versa is studied using system of differential equations. In this way the trajectory of glider is predicted as in Figure 3.2. The vertical height of glider increase for sometime and then start decreasing. The horizontal distance travelled by glider keep on increasing until it reaches ground. The interaction and long term behaviour is also studied. It is determined that how the solutions in the neighbourhood of the equilibrium points behave. The type of equilibrium point comes out to be stable node (see figure 3.3). Thus the glider flies with constant scalar velocity at constant angle after it reaches stable point. The glider must be kept slightly downwards from front in order to reach stable condition as soon as possible (see figure 4.1).

The glider model has been calculated and studied in this report. Some as-

sumptions were required to be made for initiating the modelling. The effects of turning, bad weather conditions and other influencing factors were not considered. So, this model need some improvements which can be made in future study.

Bibliography

- [1] Glider (aircraft). Wikipedia. Retrieved on 24 June,2017 from [https://en.wikipedia.org/wiki/Glider_\(aircraft\)](https://en.wikipedia.org/wiki/Glider_(aircraft)).
- [2] Hindustan Ardhra . Wikipedia. Retrieved on 24 June, 2017 from https://en.wikipedia.org/wiki/Hindustan_Ardhra.
- [3] B V Ramana. (2008). Higher Engineering Mathematics: Vectors (Sixth ed.). New Delhi: McGraw Hill.

Appendix A

Python Code for $\frac{d\theta}{dt}$

```
import numpy as np
import matplotlib.pyplot as plt

print("Solution for d\u03B8/dt") # in Python 2.7: use no brackets

# Initializations

Dt = 1/4                                # timestep Delta t
theta_init = np.pi / 6                  # initial angle
t_init = 0                               # initial time
t_end = 100                             # stopping time
n_steps = int(round((t_end-t_init)/Dt)) # total number of timesteps

t_arr = np.zeros(n_steps + 1)           # create an array of zeros for t
theta_arr = np.zeros(n_steps + 1)       # create an array of zeros for theta
t_arr[0] = t_init                        # add the initial P to the array
theta_arr[0] = theta_init                # add the initial t to the array

# Euler's method

for i in range(1, n_steps + 1):
    theta = theta_arr[i-1]
    t = t_arr[i-1]
    dthetadt = (3*27.78*27.78 - 470*9.81*np.cos(theta)) / (470*27.78)
    theta_arr[i] = theta + Dt*dthetadt # calculate theta on next time step
    t_arr[i] = t + Dt                  # adding the new t-value to the list

# Plot the results
```

```

fig = plt.figure()                                # create figure
plt.plot(t_arr, theta_arr, linewidth = 4)          # plot angle vs. time

plt.title('Graph of  $d\theta/dt$  for  $dt=1/4$  sec,  $\theta(0)=6\text{rad}$ ', fontsize =15)
plt.xlabel('t (in sec)', fontsize = 15)
plt.ylabel(' $\theta(t)$ ', fontsize = 15)

plt.xticks(fontsize = 15)
plt.yticks(fontsize = 15)
plt.grid(True)                                    # show grid
plt.axis([0, 25, -2, 2])                          # define the axes
plt.show()                                         # show the plot
# save the figure as .jpg
fig.savefig('Glider2_4.jpg', dpi=fig.dpi, bbox_inches = "tight")

```

Appendix B

Python Code for $\frac{dV}{dt}$

```
import numpy as np
import matplotlib.pyplot as plt

print("Solution for dV/dt") # in Python 2.7: use no brackets

# Initializations

Dt = 1/32                                # timestep Delta t
V_init = 50                              # initial velocity
t_init = 0                               # initial time
t_end = 300                              # stopping time
n_steps = int(round((t_end-t_init)/Dt)) # total number of timesteps

t_arr = np.zeros(n_steps + 1)            # create an array of zeros for t
V_arr = np.zeros(n_steps + 1)            # create an array of zeros for V
t_arr[0] = t_init                         # add the initial V to the array
V_arr[0] = V_init                         # add the initial t to the array

# Euler's method

for i in range(1, n_steps + 1):
    V = V_arr[i-1]
    t = t_arr[i-1]
    dVdt = (-13.90*V*V + 470*9.81* np.sin(1.045) ) / 470
    V_arr[i] = V + Dt*dVdt                # calculate V on the next time step
    t_arr[i] = t + Dt                     # adding the new t-value to the list

# Plot the results
```

```

fig = plt.figure()                # create figure
plt.plot(t_arr, V_arr, linewidth = 4)  # plot velocity vs. time

plt.title('dV/dt for dt=1/32 sec, V(0)=50 m/s', fontsize = 15)
plt.xlabel('t (in sec)', fontsize = 15)
plt.ylabel('V(t)', fontsize = 15)

plt.xticks(fontsize = 15)
plt.yticks(fontsize = 15)
plt.grid(True)                    # show grid
plt.axis([0, 10, 0, 60])         # define the axes
plt.show()                       # show the plot
# save the figure as .jpg
fig.savefig('Glider2_4b.jpg', dpi=fig.dpi, bbox_inches = "tight")

```


Appendix C

Python Code for System of Two Differential Equations

```
import numpy as np
import matplotlib.pyplot as plt

print("Solution for  $dV/dt$  ,  $d\theta/dt$  ")

# Initializations

Dt = 1/32                                # timestep Delta t
V_init = 50                              # initial value of V
theta_init = np.pi/6                    # initial value of theta
t_init = 0                               # initial time
t_end = 100                              # stopping time

n_steps = int(round((t_end-t_init)/Dt)) # total number of timesteps

X = np.zeros(2)                          # create space for current  $X=[V,\theta]^T$ 
dXdt = np.zeros(2)                      # create space for current derivative
t_arr = np.zeros(n_steps + 1)           # create a storage array for t
X_arr = np.zeros((2,n_steps+1))         # create storage array for  $X=[V,\theta]^T$ 
t_arr[0] = t_init                        # add the initial t to the storage array
X_arr[0,0] = V_init                      # add the initial V to the storage array
X_arr[1,0] = theta_init                  # add the initial theta to the storage array

# Euler's method

for i in range (1, n_steps + 1):
    t = t_arr[i-1]                      # load the time
```

```

V = X_arr[0,i-1]                # load the value of V
theta = X_arr[1,i-1]            # load the value of theta
X[0] = V                        # fill current state vector X=[V,theta]^T
X[1] = theta
dVdt = (-13.90*V*V - 470*9.81*np.sin(theta) ) / 470
dthetadt = (3*V*V - 470*9.81*np.cos(theta) ) / (470*V)
dXdt[0] = dVdt                  # fill derivative vector dX/dt
dXdt[1] = dthetadt
Xnew = X + Dt*dXdt              # calculate X on next time step
X_arr[:,i] = Xnew               # store Xnew
t_arr[i] = t + Dt               # store new t-value

# Plot the results

fig = plt.figure()
plt.plot(t_arr, X_arr[0,:], linewidth = 4, label="V(t)") # plot V vs. time
plt.plot(t_arr, X_arr[1,:], linewidth = 4, label="\u03B8(t)")

plt.title('dV/dt & d\u03B8/dT for dt=1/32s' , fontsize = 15) # set title
plt.xlabel('t (in seconds)', fontsize = 15) # name of horizontal axis
plt.ylabel('V(t) and \u03B8(t)', fontsize = 15) # name of vertical axis

plt.xticks(fontsize = 15)        # adjust the fontsize
plt.yticks(fontsize = 15)        # adjust the fontsize
plt.axis([0, 10, -20, 60])      # set the range of the axes

plt.legend(fontsize=15)          # show the legend
plt.show()                       # necessary for some platforms

# save the figure as .jpg (other formats: png, pdf, svg, (ps, eps))
fig.savefig('Glider3_1.jpg', dpi=fig.dpi, bbox_inches = "tight")

```

Appendix D

Python Code for System of Four Differential Equations

```
import numpy as np
import matplotlib.pyplot as plt

print("Solution for dV/dt, d\u03B8/dt, dx/dt, dy/dt")

# Initializations

Dt = 1/32                                # timestep Delta t
V_init = 50
theta_init = np.pi/6
x_init = 0
y_init = 1000
t_init = 0                                # initial time
t_end = 100                               # stopping time

n_steps = int(round((t_end-t_init)/Dt)) # total number of timesteps

S = np.zeros(4)
dSdt = np.zeros(4)                       # create space for current derivative
t_arr = np.zeros(n_steps + 1)            # create a storage array for t
S_arr = np.zeros((4,n_steps+1))          # create a storage array
t_arr[0] = t_init                         # add the initial t to the storage array
S_arr[0,0] = V_init                       # add the initial V to the storage array
S_arr[1,0] = theta_init
S_arr[2,0] = x_init
S_arr[3,0] = y_init
```

```

# Euler's method

for i in range (1, n_steps + 1):
    t = t_arr[i-1]                # load the time
    V = S_arr[0,i-1]              # load the value of V
    theta = S_arr[1,i-1]
    x = S_arr[2,i-1]
    y = S_arr[3,i-1]
    S[0] = V
    S[1] = theta
    S[2] = x
    S[3] = y
    dVdt = (-13.90*V*V - 470*9.81*np.sin(theta) ) / 470
    dthetadt = (3*V*V - 470*9.81*np.cos(theta) ) / (470*V)
    dxdt = V*np.cos(theta)
    dydt = V*np.sin(theta)        # calculate the derivative dtheta/dt
    dSdt[0] = dVdt                # fill derivative vector dX/dt
    dSdt[1] = dthetadt
    dSdt[2] = dxdt
    dSdt[3] = dydt
    Snew = S + Dt*dSdt            # calculate X on next time step
    S_arr[:,i] = Snew            # store Xnew
    t_arr[i] = t + Dt            # store new t-value

# Plot the results

fig = plt.figure()
plt.plot(t_arr, S_arr[0,:], linewidth = 4, label="V(t)")
plt.plot(t_arr, S_arr[1,:], linewidth = 4, label="\u03B8(t)")
plt.plot(t_arr, S_arr[2,:], linewidth = 4, label="x(t)")
plt.plot(t_arr, S_arr[3,:], linewidth = 4, label="y(t)")

plt.title('dV/dt, d\u03B8/dT, dx/dt, dy/dt' , fontsize = 20) # set title
plt.xlabel('t (in seconds)', fontsize = 20) # name of horizontal axis
plt.ylabel('V(t), \u03B8(t), x(t), y(t)', fontsize = 20) # name of axis

plt.xticks(fontsize = 15)        # adjust the fontsize
plt.yticks(fontsize = 15)        # adjust the fontsize
plt.axis([0, 70, -20, 1100])    # set the range of the axes

plt.legend(fontsize=15)          # show the legend
plt.show()                      # necessary for some platforms

# save the figure as .jpg (other formats: png, pdf, svg, (ps, eps))
fig.savefig('Glider3_2.jpg', dpi=fig.dpi, bbox_inches = "tight")

```


Appendix E

Python Code for Different Initial Conditions of $\theta(t)$

```
import numpy as np
import matplotlib.pyplot as plt

print("Solution for dV/dt, d\u03B8/dt, dx/dt, dy/dt")

# Initializations

Dt = 1/32                                # timestep Delta t
theta1_init = np.pi/2                   # initial values
theta2_init = np.pi/2.312
theta3_init = np.pi/2.313
theta4_init = np.pi/3
theta5_init = 0
theta6_init = -np.pi/2
t_init = 0                               # initial time
t_end = 200                             # stopping time

n_steps = int(round((t_end-t_init)/Dt)) # total number of timesteps

S = np.zeros(6)                          # create space
dSdt = np.zeros(6)                       # create space for current derivative
t_arr = np.zeros(n_steps + 1)            # create a storage array for t
S_arr = np.zeros((6,n_steps+1))          # create a storage array
t_arr[0] = t_init                         # add the initial t to the storage array
S_arr[0,0] = theta1_init                  # add the initial values to the storage array
S_arr[1,0] = theta2_init
```

```

S_arr[2,0] = theta3_init
S_arr[3,0] = theta4_init
S_arr[4,0] = theta5_init
S_arr[5,0] = theta6_init

# Euler's method

for i in range(1, n_steps + 1):
    t = t_arr[i-1]                # load the time
    theta1 = S_arr[0,i-1]         # load the value
    theta2 = S_arr[1,i-1]
    theta3 = S_arr[2,i-1]
    theta4 = S_arr[3,i-1]
    theta5 = S_arr[4,i-1]
    theta6 = S_arr[5,i-1]         # load the value
    S[0] = theta1                 # fill current state vector
    S[1] = theta2
    S[2] = theta3
    S[3] = theta4
    S[4] = theta5
    S[5] = theta6
    dtheta1dt = (3*18*18 - 470*9.81*np.cos(theta1)) / (470*18) # calculate the der
    dtheta2dt = (3*18*18 - 470*9.81*np.cos(theta2)) / (470*18)
    dtheta3dt = (3*18*18 - 470*9.81*np.cos(theta3)) / (470*18)
    dtheta4dt = (3*18*18 - 470*9.81*np.cos(theta4)) / (470*18)
    dtheta5dt = (3*18*18 - 470*9.81*np.cos(theta5)) / (470*18)
    dtheta6dt = (3*18*18 - 470*9.81*np.cos(theta6)) / (470*18)
    dSdt[0] = dtheta1dt           # fill derivative vector
    dSdt[1] = dtheta2dt
    dSdt[2] = dtheta3dt
    dSdt[3] = dtheta4dt
    dSdt[4] = dtheta5dt
    dSdt[5] = dtheta6dt
    Snew = S + Dt*dSdt
    S_arr[:,i] = Snew              # store Xnew
    t_arr[i] = t + Dt             # store new t-value

# Plot the results

fig = plt.figure()
plt.plot(t_arr, S_arr[0,:], linewidth = 4, label="\u03B81(t)") # plot graph
plt.plot(t_arr, S_arr[1,:], linewidth = 4, label="\u03B82(t)")
plt.plot(t_arr, S_arr[2,:], linewidth = 4, label="\u03B83(t)")
plt.plot(t_arr, S_arr[3,:], linewidth = 4, label="\u03B84(t)")
plt.plot(t_arr, S_arr[4,:], linewidth = 4, label="\u03B85(t)")

```

```

plt.plot(t_arr, S_arr[5:], linewidth = 4, label="\u03B86(t)")

plt.title('\u03B81(0)=/2, \u03B82(0)=/2.312, \u03B83(0)=/2.313, \u03B84(0)=/3, \u03B85(0)=/4, \u03B86(0)=/5')
plt.xlabel('t (in seconds)', fontsize = 15)    # name of horizontal axis
plt.ylabel('\u03B81(t), \u03B82(t), \u03B83(t), \u03B84(t), \u03B85(t), \u03B86(t)',
           fontsize = 15)

plt.xticks(fontsize = 12)                    # adjust the fontsize
plt.yticks(fontsize = 12)                    # adjust the fontsize
plt.axis([0, 30, -4, 8])                     # set the range of the axes

plt.legend(fontsize=12)                      # show the legend
plt.show()                                  # necessary for some platforms

# save the figure as .jpg (other formats: png, pdf, svg, (ps, eps))
fig.savefig('Glider4_1.jpg', dpi=fig.dpi, bbox_inches = "tight")

```


Appendix F

Python Code for Different Initial Conditions

```
import numpy as np
import matplotlib.pyplot as plt

print("Solution for  $dV/dt$  ,  $d\theta/dt$  ")

# Initializations

Dt = 1/32                                # timestep Delta t
V1_init = 500
V2_init = 50
V3_init = 5                              # initial value of V
theta1_init = 0
theta2_init = 0
theta3_init = 0                          # initial value of theta
t_init = 0                               # initial time
t_end = 100                              # stopping time

n_steps = int(round((t_end-t_init)/Dt)) # total number of timesteps

X = np.zeros(6)                          # create space for current
dXd_t = np.zeros(6)                     # create space for current derivative
t_arr = np.zeros(n_steps + 1)           # create a storage array for t
X_arr = np.zeros((6,n_steps+1))         # create a storage array
t_arr[0] = t_init                        # add the initial t to storage array
X_arr[0,0] = V1_init
X_arr[1,0] = V2_init
X_arr[2,0] = V3_init                     # add the initial V to the storage array
```

```

X_arr[3,0] = theta1_init
X_arr[4,0] = theta2_init
X_arr[5,0] = theta3_init

# Euler's method

for i in range(1, n_steps + 1):
    t = t_arr[i-1]                # load the time
    V1 = X_arr[0,i-1]
    V2 = X_arr[1,i-1]
    V3 = X_arr[2,i-1]             # load the value of V
    theta1 = X_arr[3,i-1]
    theta2 = X_arr[4,i-1]
    theta3 = X_arr[5,i-1]         # load the value of theta
    X[0] = V1
    X[1] = V2
    X[2] = V3
    X[3] = theta1
    X[4] = theta2
    X[5] = theta3
    dV1dt = (-13.90*V1*V1 - 470*9.81*np.sin(theta1)) / 470
    dV2dt = (-13.90*V2*V2 - 470*9.81*np.sin(theta2)) / 470
    dV3dt = (-13.90*V3*V3 - 470*9.81*np.sin(theta3)) / 470
    dtheta1dt = (3*V1*V1 - 470*9.81*np.cos(theta1)) / (470*V1)
    dtheta2dt = (3*V2*V2 - 470*9.81*np.cos(theta2)) / (470*V2)
    dtheta3dt = (3*V3*V3 - 470*9.81*np.cos(theta3)) / (470*V3)
    dXdt[0] = dV1dt
    dXdt[1] = dV2dt
    dXdt[2] = dV3dt               # fill derivative vector dX/dt
    dXdt[3] = dtheta1dt
    dXdt[4] = dtheta2dt
    dXdt[5] = dtheta3dt
    Xnew = X + Dt*dXdt            # calculate X on next time step
    X_arr[:,i] = Xnew             # store Xnew
    t_arr[i] = t + Dt            # store new t-value

# Plot the results

fig = plt.figure()
plt.plot(t_arr, X_arr[0,:], linewidth = 4, label="V1(t)")
plt.plot(t_arr, X_arr[1,:], linewidth = 4, label="V2(t)")
plt.plot(t_arr, X_arr[2,:], linewidth = 4, label="V3(t)") # plot V vs. time
plt.plot(t_arr, X_arr[3,:], linewidth = 4, label="\u03B81(t)") # plot theta vs. time
plt.plot(t_arr, X_arr[4,:], linewidth = 4, label="\u03B82(t)")
plt.plot(t_arr, X_arr[5,:], linewidth = 4, label="\u03B83(t)")

```

```

plt.title('dV/dt & d\u03B8/dT for \u03B8(0)=0' , fontsize = 13) # set title
plt.xlabel('t (in seconds)', fontsize = 13) # name of horizontal axis
plt.ylabel('V(t) and \u03B8(t)', fontsize = 13) # name of vertical axis

plt.xticks(fontsize = 13) # adjust the fontsize
plt.yticks(fontsize = 13) # adjust the fontsize
plt.axis([0, 6, -5, 100]) # set the range of the axes

plt.legend(fontsize=15) # show the legend
plt.show() # necessary for some platforms

# save the figure as .jpg (other formats: png, pdf, svg, (ps, eps))
fig.savefig('Glider4_2b.jpg', dpi=fig.dpi, bbox_inches = "tight")

```